



Technical Documentation

Dm verity Metadata Generation

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

1. Introduction

1.1. Purpose

This document provides technical specifications and operational details for the `generate_verity.sh` script. This script executes the final, multi-stage process of creating a cryptographically verifiable root filesystem image by generating the dm-verity Merkle hash tree and integrating signed metadata.

The script implements a highly secure configuration using a whole-disk approach (no partition table) with a detached PKCS7 signature verified by the kernel's trusted keyring.

1.2. Background and Architecture

This script is executed immediately after `build_rootfs.sh`. Its primary function is to fill the slack space reserved at the end of the `rootfs.img` with the dm-verity hash tree and the necessary metadata components (signed header, signature, and locator footer). The goal is to enable the kernel to automatically verify the integrity of the entire root filesystem before mounting it.

1.3. Output and Result Location

The script generates several critical artifacts, which are all placed within a dedicated metadata directory relative to the build script's location:

- **Metadata Directory:** `$BUILD\_DIR/Binaries/metadata` (defined as `$META\_DIR` in the script).

This directory contains the core verification files required by the subsequent boot process:

- **root.hash:** Contains the final Merkle root hash.
- **verity_header.bin:** The 196-byte signed metadata header.
- **verity_header.sig:** The detached PKCS7 signature.
- **verity_locator.bin:** The 4 KiB VLOC locator footer
- **verity_info.txt:** Detailed output from the `veritysetup` format command.

2. Script Specification: `generate_verity.sh`

2.1. Overview

The script is a 9-stage pipeline that combines standard Linux utilities (`losetup`, `dumpe2fs`, `veritysetup`, `openssl`) with custom Python logic for geometry calculation, binary structure packing, and direct disk writing. It relies on cryptographic keys and certificates located in the project's relative path (`../boot/`).

2.2. Critical Stages and Technical Flow

The process focuses on precisely calculating and writing four components to the end of the disk image: the Hash Tree, the Metadata Header, the Signature, and the Locator Footer.

Stage [1/9] & [2/9]: Loop Setup and Geometry

The script attaches the rootfs.img to a loop device (\$LOOP_DEV) and uses dumpe2fs to read the exact Block Size and Block Count of the ext4 filesystem. This geometry information is essential for calculating the Merkle tree size. The script works directly on the whole disk device (\$DISK_DEV).

Stage [3/9]: Merkle Tree Sizing and Filesystem Shrink

This is the most critical geometry step. It uses a Python block to mathematically estimate the required size of the dm-verity Merkle hash tree (using sha256 and the filesystem geometry).

The Python logic estimates the required space. If the available slack space is insufficient, the script uses “e2fsck” and “resize2fs” to shrink the ext4 filesystem and free up the exact space needed for the hash tree plus a safety margin:

```
104 SAFETY=$((1<<20)) # 1 MiB slack
105 TARGET_BYTES=$((DISK_SIZE - HASH_BYTES - SAFETY))
106 TARGET_BLOCKS=$((TARGET_BYTES / BLOCK_SIZE))
107 if (( TARGET_BLOCKS <= 0 )); then
108     echo "FATAL: disk too small for data + verity tree"; exit 1
109 fi
110
111 echo "  resize2fs $DISK_DEV $TARGET_BLOCKS"
112 sudo resize2fs "$DISK_DEV" "$TARGET_BLOCKS"
```

Stage [4/9] & [5/9]: Hash Tree Generation

The veritysetup format command is used to generate the Merkle hash tree directly onto the disk image, starting at the calculated hash offset (\$HASH_OFFSET).

- **Key Parameter:** The --no-superblock flag is used, confirming the whole-disk, no-superblock architecture.

```
128 sudo veritysetup format \
129     --data-block-size="$BLOCK_SIZE" \
130     --hash-block-size="$BLOCK_SIZE" \
131     --data-blocks="$BLOCK_COUNT" \
132     --hash-offset="$HASH_OFFSET" \
133     --no-superblock \
134     "$DISK_DEV" "$DISK_DEV" \
135     | tee "$META_DIR/verity_info.txt"
```

The command output is parsed to extract the Root Hash (\$ROOT_HASH) and Salt (\$SALT_HEX).

Stage [6/9]: Metadata Header Construction

A Python script uses the struct library to precisely pack a 196-byte binary metadata header (\$METADATA_HEADER). This header contains all parameters needed by the kernel, including the Root Hash, Salt, and geometry details.

Stage [7/9]: Detached Signature Creation

The unsigned metadata header is cryptographically signed using the provided private key and certificate via openssl smime. This creates a PKCS7 detached signature (\$SIG_FILE):

```
179  openssl smime -sign -binary -noattr \  
180      -in "$METADATA_HEADER" \  
181      -signer "$CERT_FILE" -inkey "$PRIV_KEY" \  
182      -outform DER -nosmimecap > "$SIG_FILE"  
183  SIG_SIZE=$(stat -c%s "$SIG_FILE")
```

Stage [8/9]: Final Disk Writing and Layout

A privileged Python script (run via sudo) manages the final, precise binary write operations directly to the disk image. It calculates the final layout from the end of the disk backward, ensuring all components are properly aligned (4 KiB) and positioned:

1. **VLOC Locator Footer (4 KiB):** Written to the very end of the disk (\$LOCATOR_OFFSET).
2. **PKCS7 Signature:** Written before the VLOC, padded to 4 KiB alignment (\$SIG_OFFSET).
3. **Metadata Header:** Written before the Signature, padded to 4 KiB alignment (\$META_OFFSET).

The absolute offsets are also exported to a shell file (\$OFFSETS_FILE) for logging.

3. Security and Architecture Summary

The generated disk image adheres to a strict on-disk layout, guaranteeing the kernel can locate and verify the necessary components:

[ext4 data | Merkle Hash Tree | Metadata Header Region | PKCS#7 Signature Region | VLOC Locator]

This architecture ensures the dm-verity kernel module can perform verification against a trusted signature, thereby enforcing the integrity of the entire root filesystem.

End of Document